



Compilador de asm Z80 a asm x86-64

Diseño e implementación

1. Equipo	2
2. Repositorio	2
3. Dominio	2
4. Construcciones	3
5. Casos de Prueba	3
6. Ejemplos	4

1. Equipo

Nombre	Apellido	Legajo	E-mail
Nahuel Ignacio	Prado	64276	naprado@itba.edu.ar
Lorenzo	Chiossone	64359	lchiossone@itba.edu.ar
Mateo	Buela	64680	mbuela@itba.edu.ar

2. Repositorio

El correspondiente repositorio para la realización del proyecto se encuentra en el siguiente [link](#).

3. Dominio

Desarrollar un compilador que traduzca programas escritos en el lenguaje ensamblador del procesador Zilog Z80 al lenguaje ensamblador x86-64 de Intel/amd. El compilador debe modelar con precisión las capacidades y limitaciones de arquitectura de los procesadores Z80, asegurando que cualquier instrucción inválida para dicho procesador sea detectada y genere un error de compilación.

Por otro lado se desprecian ciertas sentencias obsoletas como **ORG** o **ASEG**, que carecen de equivalencia directa o relevancia en el entorno x86-64 moderno.

Se debe mantener la lógica de las flags así mantener la coherencia entre los códigos pre y post compilación, aunque esto no quiere decir que estas se mantengan con el mismo nombre, ya que se adaptan al nuevo lenguaje.

El compilador también debe mantener soporte para la creación y uso de macros, permitiendo que el programador defina secuencias reutilizables de instrucciones que sean correctamente traducidas al conjunto objetivo.

Gracias a esta solución, se podrá migrar y reutilizar código del Z80 hacia plataformas modernas de 64 bits.

4. Construcciones

El compilador desarrollado debería ofrecer las siguientes funcionalidades:

- I. Traducción completa de instrucciones Z80 a equivalentes x86-64. Se soportará todas las instrucciones aritméticas, lógicas, de transferencia de datos, control de flujo y manipulación de bits del Z80, generando código x86-64 que preserve su semántica.
- II. Validación estricta de operaciones según la arquitectura Z80 en modos de direccionamiento, tipos de registros y tamaños de operandos válidos para el Z80, emitiendo errores en caso de usar combinaciones ilegales.
- III. Mantenimiento y adaptación de flags del Z80 (S, Z, H, P/V, N, C). Garantizando coherencia en comparaciones y saltos condicionales.
- IV. Soporte para pseudoinstrucciones y directivas de ensamblado como EQU, DEFL, DB, DW, DS, PUBLIC, EXTERN, eliminando o reemplazando aquellas sin sentido en x86-64 como ASEG u ORG.
- V. Traducción de modos de direccionamiento Z80. Inclusión de mecanismos para traducir direccionamiento inmediato, directo, indirecto, indexado (IX/IY + desplazamiento) y relativo a las formas equivalentes en x86-64.
- VI. Soporte de macros. Capacidad para definir macros en Z80 que se expandan y traduzcan correctamente a secuencias equivalentes en x86-64.
- VII. Compatibilidad con segmentos de datos y código. Traducción y reorganización de CSEG, DSEG y otras directivas de segmentación a secciones .text y .data en x86-64.
- VIII. Emisión de advertencias. Avisos sobre instrucciones obsoletas, no estándar o extensiones del Z80 que no tengan equivalente directo.
- IX. Opciones de salida flexibles. Posibilidad de generar como salida código ensamblador x86-64 legible, código objeto o ejecutables directamente.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- I. Se deben poder hacer una suma directa de 8 bits con el registro A.
- II. Se debe poder hacer un load del registro HL a SP.
- III. Se deben poder hacer una suma indexada de 8 bits con el registro A.
- IV. Se debe poder hacer un load de una constante a HL.

- V. Se deben poder hacer una suma de 16 bits con el registro HL.
- VI. Se deben poder hacer una resta indirecta de 8 bits con el registro A.
- VII. Se debe poder hacer un incremento en el registro B.
- VIII. Se debe poder hacer un incremento en el registro C.
- IX. Se debe poder negar A.
- X. Se debe poder hacer un load de una constante a D.

Se proponen los siguientes casos iniciales de prueba de **rechazo**:

- I. Hacer una suma al registro B.
- II. Hacer un “AND” entre B y C.
- III. Hacer un complemento a 1 en el registro E.
- IV. Hacer un load de registro indirecto a otro registro indirecto.
- V. Hacer un push con el registro A.

6. Ejemplos

```
start:      ld IX, numero1      ; cargo la dirección de numero1 en
IX
            ld A, (numero2)     ; cargo el valor de numero2 (8) en A
            add A, (IX)         ; sumo a A (8) lo que hay en numero1
(7)
            ld (respuesta), A   ; bajo a la dirección respuesta
                                ; el valor de A (15)
            rst 38h

numero1:    db 7                ; define un byte llamado numero1
                                ; con el valor 7
numero2:    db 8                ; define un byte llamado numero2
                                ; con el valor 8
respuesta:  ds 1                ; define un byte llamado respuesta
                                ; sin valor definido

            end start
```

```
start:      ld IX, vector       ; carga en IX la dirección de vector
            ld A, inicial       ; carga en A el valor inicial (60)
            ld B, cantidad      ; cargo en B la cantidad de elementos

ciclo:      sub A, (IX)         ; le resto a A el valor en la
                                ; dirección IX
            inc IX              ; incremento IX (apuntando
                                ; al siguiente valor del vector)
            djnz ciclo          ; resto 1 a B y si no dió 0 salto a
ciclo
            ld (respuesta), A   ; cargo el valor de A (20) en
respuesta
```

```

        rst 38h                ; fin del programa

inicial: equ 60                ; defino la constante inicial
                                ; con el valor 60
vector:  db 10, 20, 5, 3, 2    ; defino un vector de 5 bytes
cantidad: equ $ - vector       ; defino el valor cantidad
                                ; con el tamaño del vector.
respuesta: ds 1                ; defino un byte para respuesta

        end start

```

```

start:   ld HL, 1000h           ; HL = dirección del mensaje
        ld C, 1020h - 1000h    ; C = cantidad de caracteres
                                ; del mensaje

ciclo:   ld A, (HL)

        ; Se analiza si el carácter está entre 'A' y 'Z', o sea si es una
        ; mayúscula, en cuyo caso se pasa a minúscula, sinó se deja como
        ; estaba

        cp 'A'                 ; Si es menor que la letra
        jp M, sigo             ; 'A', no se cambia
        cp 'Z' + 1             ; Si es mayor que la letra
        jp P, sigo             ; 'Z', no se cambia

        ; Si se llegó hasta acá es porque es mayúscula
        add A, 'f'-'F'         ; entonces se pasa a minúscula
        ld (HL), A             ; y se copia en el nuevo mensaje

sigo:    inc HL                 ; Se desplaza al siguiente carácter
        dec C
        jr NZ, ciclo
        rst 38h                ; Fin del programa
        end start

```

```

        ld A, (1000h)
        ld B, A
        inc B                  ; Se incrementa B para "comenzar"
                                ; el ciclo en la instrucción djnz

        ld C, 0                ; C será acumulador de la suma parcial
        ld HL, 1001h           ; HL apunta al comienzo del vector

ciclo:   jp calculo

        ld A, (HL)             ; Carga en A el valor apuntado por HL
        cp 0                   ; Analiza su signo
        jp M, otro             ; Si es negativo se "saltea"
        add A, C               ; Acumula el valor
        ld C, A                ; Resguarda la suma parcial en C

otro:    inc HL                 ; Próximo elemento

calculo: djnz ciclo
        ld A, C
        ld (0FFFh), A          ; Almacena el resultado
        rst 38h                ; Fin del programa

```

```

        ld A, (1000h)
        ld B, A
        inc B                  ; Se incrementa B para "comenzar"

```

```

                                ; el ciclo en la instrucción djnz
                                ; C será el contador de negativos
                                ; HL apunta al comienzo del vector
                                ; Carga en A el valor apuntado por HL
                                ; Analiza su signo
                                ; Si es positivo se "saltea"
                                ; Incrementa el contador de negativos
                                ; Próximo elemento
                                ; Almacena el resultado
                                ; Fin del programa
ld      C, 0
ld      HL, 1001h
jp      calculo
ciclo:  ld      A, (HL)
cp      0
jp      P, noInc
inc     C
noInc:  inc     HL
cálculo: djnz   ciclo
ld      A, C
ld      (0FFFh), A
rst 38h

```

```

                                ; Carga la dirección inicial del string
                                ; Inicializa el contador de caracteres
                                ; Lee un carácter
                                ; Si es 00h, terminó el string
                                ; Incrementa el contador de caracteres
                                ; Se desplaza al siguiente carácter
                                ; Fin
ld      IX, 2000h
ld      B, 0
ciclo:  ld      A, (IX)
cp      0
jp      Z, fin
inc     B
inc     IX
jp      ciclo
fin:    rst 38h

```

```

                                ; Carga la dirección inicial del string
                                ; Inicializa el contador de caracteres
                                ; Lee un carácter
                                ; Si es 00h, terminó el string
                                ; Si no es un espacio se debe
                                ; "saltar" el incremento
                                ; Incrementa el contador de espacios
                                ; Se desplaza la siguiente carácter
                                ; Fin
ld      IX, 2000h
ld      C, 0
ciclo:  ld      A, (IX)
cp      0
jp      Z, fin
cp      ' '
jp      NZ, noInc
inc     C
noInc:  inc     IX
jp      ciclo
fin:    rst 38h

```

```

                                ; IX = dirección inicial del string 1
                                ; IY = dirección inicial del string 2
                                ; Lee un carácter del string 1
                                ; Lo compara con el correspondiente
                                ; del string 2
                                ; El carácter del string 1 es igual al carácter del string 2
                                ; Si el caracater es 00h, terminaron
                                ; ambos strings y son iguales
                                ; Se desplaza en el string 1
                                ; se desplaza en el string 2
                                ; FinSi
                                ; 80h + 80h: enciende el carry
                                ; cualquier valor + 0: apaga el carry
ld      IX, 2000h
ld      IY, 2100h
ciclo:  ld      A, (IX)
cp      (IY)
jp      NZ, finNo
; El carácter del string 1 es igual al carácter del string 2
cp      0
jp      Z, finSi
inc     IX
inc     IY
jp      ciclo
finSi:  ld      A, 80h
add     A, 80h
jp      fin
finNo:  add     A, 0

```

```
fin:      rst 38h
```